

---

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> repr(purse) # Calls WizCoin's __repr__() behind the scenes.
'WizCoin(2, 5, 10)'
>>> str(purse) # Calls WizCoin's __str__() behind the scenes.
'2g, 5s, 10k'
>>> print(f'My purse contains {purse}.') # Calls WizCoin's __str__().
My purse contains 2g, 5s, 10k.
```

---

When we pass the `WizCoin` object in `purse` to the `repr()` and `str()` functions, behind the scenes Python calls the `WizCoin` class's `__repr__()` and `__str__()` methods. We programmed these methods to return more readable and useful strings. If you entered the text of the `'WizCoin(2, 5, 10)'` repr string into the interactive shell, it would create a `WizCoin` object that has the same attributes as the object in `purse`. The `str` string is a more human-readable representation of the object's value: `'2g, 5s, 10k'`. If you use a `WizCoin` object in an f-string, Python uses the object's `str` string.

If `WizCoin` objects were so complex that it would be impossible to create a copy of them with a single constructor function call, we would enclose the repr string in angle brackets to denote that it's not meant to be Python code. This is what the generic representation strings, such as `'<wizcoin.WizCoin object at 0x00000212B4148EE0>'`, do. Typing this string into the interactive shell would raise a `SyntaxError`, so it couldn't possibly be confused for Python code that creates a copy of the object.

Inside the `__repr__()` method, we use `self.__class__.__qualname__` instead of hardcoding the string `'WizCoin'`; so if we subclass `WizCoin`, the inherited `__repr__()` method will use the subclass's name instead of `'WizCoin'`. In addition, if we rename the `WizCoin` class, the `__repr__()` method will automatically use the updated name.

But the `WizCoin` object's `str` string shows us the attribute values in a neat, concise form. I highly recommend you implement `__repr__()` and `__str__()` in all your classes.